Day 4

Data Structures

Lists, Tuples, Dictionaries, Sets and Comprehensions

Instructor Session Guide

2 hours	Beginner	4 concept files	Days 1-3
Session length	Phase	Student files	Prerequisites

Learning Objectives

- Create and change lists, and avoid the reference-copy trap
- Explain why tuples are immutable and use packing and unpacking
- Store and look up data with dictionaries, safely, using .get()
- Use sets for unique items and set operations (union, intersection, difference)
- Slice lists and strings with start:stop:step
- Write list, dict and set comprehensions

Student files for this session: day04_01_lists, day04_02_tuples, day04_03_dictionaries, day04_04_sets_comprehensions.

SECTION 1 - SESSION OVERVIEW

Session Goal

Introduce Python's four core collection types and teach students to choose the right one for a given problem, then show how comprehensions build new collections in one readable line.

Session Outcome

Students can run all four concept files, finish each mini-challenge, and explain in one sentence when to use a list, tuple, dict or set.

Timeline

Time	Activity	Type
0:00-0:10	Welcome, recap of Days 1-3, preview of the four collection types	Intro
0:10-0:35	Lists: creation, mutation, sorting, slicing, copying (file 1)	Teach + live code
0:35-0:50	Tuples: immutability, packing, unpacking (file 2)	Teach + live code
0:50-1:15	Dictionaries: key-value pairs, .get(), iteration, nesting (file 3)	Teach + live code
1:15-1:30	Sets and set operations (file 4, part 1)	Teach + live code
1:30-1:50	List, dict and set comprehensions (file 4, part 2)	Teach + live code
1:50-2:00	Wrap-up, checklist, Day 5 preview	Wrap-up

Instructor Note

Anchor each type with one analogy and return to it when students get lost:

List = grocery list (ordered, changeable). Tuple = GPS coordinates (fixed once set).

Dictionary = phonebook (look up by name). Set = guest list where each name appears once.

SECTION 2 - PREREQUISITES AND PREPARATION

Student Prerequisites

Concept	Day	Why it matters today
Variables and data types	Day 2	Collections hold typed values; type() is used throughout
if / elif / else	Day 3	Filtering inside comprehensions uses conditions
for loops and range()	Day 3-4 material	Loops build lists and dicts before comprehensions replace them
Functions (basic)	Preview	File 2 returns a tuple from a small function; explain def in one line if needed

Instructor Pre-Session Checklist

- [ ] Download the four .py files and confirm each runs with: python3 filename.py
- [ ] VS Code open with the virtual environment active (Day 1 setup)
- [ ] Terminal font size large enough to read on a shared screen
- [ ] Scratch file day04_live.py open for improvising demos
- [ ] Remember: set output order can differ on each run, so warn students before file 4

Note on file contents

The student files use for loops, and file 2 uses one short function. If the course order puts functions after this day, introduce loops and def in one sentence each when they appear and move on; they are not the focus.

SECTION 3 - STEP-BY-STEP INSTRUCTION GUIDE

Step 1 - Lists: Creating, Indexing and Mutating

What to Say

"A list is an ordered, changeable collection. Think of Priya's weekly grocery list: the order is remembered, and she can add or cross off items any time. Square brackets, zero-based positions."

Live Code

```
grocery_list = ["rice", "dal", "milk", "vegetables"]
print(grocery_list[0], grocery_list[-1])
# rice vegetables

grocery_list[0] = "basmati rice"     # change by index
grocery_list.append("sugar")         # ONE item at the end
grocery_list.extend(["tea", "salt"]) # MANY items, merged in
grocery_list.remove("milk")          # remove by VALUE
popped = grocery_list.pop()          # remove + return last
grocery_list.insert(1, "ghee")       # insert at index 1
print(grocery_list)
# ['basmati rice', 'ghee', 'dal', 'vegetables', 'sugar', 'tea']
```

Common Errors

Error / symptom	Cause	Fix
IndexError: list index out of range	Index beyond len(list) - 1	Check len() first, or use a negative index
ValueError: list.remove(x): x not in list	Removing a value that is absent	Test with if x in my_list before remove()
Nested list appears after append	append(['tea','salt']) adds ONE nested item	Use extend() to merge items

Done When

Students can create a list, index forward and backward,

and explain the difference between append() and extend().

Step 2 - Sorting, Useful Methods and Slicing

What to Say

"sort() changes the list itself. Slicing is list[start:stop:step], and the stop index is exclusive: Python stops BEFORE it. The same slicing works on strings."

Live Code

```
prices = [450, 120, 899, 60, 310]
prices.sort()                  # ascending, in place
prices.sort(reverse=True)      # descending
print(len(prices), 899 in prices, prices.index(310))
# 5 True 3

localities = ["Andheri", "Bandra", "Colaba", "Dadar", "Worli"]
print(localities[1:3])    # ['Bandra', 'Colaba']
print(localities[:2])     # first two
print(localities[-2:])    # last two
print(localities[::-1])   # reversed copy
print(localities[::2])    # every second item
print("Mumbai"[1:4])      # umb
```

Common Errors

Error / symptom	Cause	Fix
Off-by-one slice	Forgetting stop is exclusive	Count: [1:3] gives indexes 1 and 2
sort() result printed as None	sort() returns None	Use sorted(list) for a new list

Done When

Students can predict the output of three different slices

before you run them.

Step 3 - The Reference-Copy Trap

What to Say

"b = a does not copy a list. Both names point at the same list in memory, so changing one changes the other. Use .copy() or [:] for a real copy."

Live Code

```
original = ["Priya", "Arjun"]
reference_copy = original          # NOT a real copy
reference_copy.append("Riya")
print(original)
# ['Priya', 'Arjun', 'Riya']      # original changed too!

real_copy = original.copy()        # or original[:]
real_copy.append("Sanjay")
print(original)
# ['Priya', 'Arjun', 'Riya']      # unchanged this time
```

Common Errors

Error / symptom	Cause	Fix
Original list changes unexpectedly	Assigned with = instead of copying	Use .copy() or [:]

Done When

Students can explain why original changed in the first demo.

Run the lists mini-challenge (monthly expenses) before moving on.

Step 4 - Tuples: Immutability, Packing and Unpacking

What to Say

"A tuple looks like a list with round brackets, but once created it cannot change. Use it for data that should stay fixed, like GPS coordinates. A single-item tuple needs a trailing comma."

Live Code

```
mumbai_coordinates = (19.0760, 72.8777)
single_item = ("Delhi",)        # tuple (note the comma)
not_a_tuple = ("Delhi")          # just a string!

try:
    mumbai_coordinates[0] = 20.0
except TypeError as error:
    print("Error caught:", error)
# Error caught: 'tuple' object does not support item assignment

name, age, city = ("Arjun", 21, "Delhi")      # unpacking
highest, *middle, lowest = (91, 88, 76, 65, 59, 40)
print(middle)
# [88, 76, 65, 59]

price_today, price_yesterday = 450, 480
price_today, price_yesterday = price_yesterday, price_today  # swap
```

Common Errors

Error / symptom	Cause	Fix
TypeError: 'tuple' object does not support item assignment	Tried to change a tuple	Build a new tuple, or use a list if it must change
ValueError: too many values to unpack	Variable count does not match tuple length	Match counts, or capture extras with *rest
('Delhi') is a str	Missing trailing comma	Write ('Delhi',)

Done When

Students can unpack a 3-item tuple and swap two variables.

Run the tuples mini-challenge (train journey) before moving on.

Step 5 - Dictionaries: Key-Value Pairs

What to Say

"A dictionary is like a phonebook: you look up a NAME, the key, to find a NUMBER, the value. Keys must be unique. You can add, update and delete entries."

Live Code

```
student_marks = {"Riya": 88, "Sanjay": 74, "Priya": 91}
print(student_marks["Riya"])
# 88

student_marks["Arjun"] = 67        # add a new key
student_marks["Sanjay"] = 79       # update existing key
del student_marks["Priya"]         # remove a key
removed = student_marks.pop("Arjun")   # remove + return value
print(student_marks)
# {'Riya': 88, 'Sanjay': 79}
```

Common Errors

Error / symptom	Cause	Fix
KeyError: 'Vikram'	Key not in the dictionary	Use .get(key, default) or test key in d
Duplicate key silently overwritten	Keys are unique; the last value wins	Use distinct keys

Done When

Students can add, update and delete a key and explain what a key is.

Step 6 - Safe Lookups, Iteration and Nesting

What to Say

"Square brackets crash on a missing key. .get() returns a fallback instead. To loop, use .items() to get key and value together."

Live Code

```
print(student_marks.get("Vikram", 0))
# 0

for name, marks in student_marks.items():
    print(f"{name} scored {marks} marks")
# Riya scored 88 marks
# Sanjay scored 79 marks

# Counting words with .get()
sentence = "mumbai is a busy city and mumbai never sleeps"
word_counts = {}
for word in sentence.split():
    word_counts[word] = word_counts.get(word, 0) + 1

# Nested dictionary
profiles = {"Riya Deshmukh": {"age": 20, "city": "Mumbai"}}
print(profiles["Riya Deshmukh"]["city"])
# Mumbai
```

Common Errors

Error / symptom	Cause	Fix
Changing a dict while looping over it	Size changes during iteration	Loop over list(d.items()) or build a new dict
TypeError: unhashable type: 'list'	A list used as a key	Use a tuple key instead

Done When

Students can loop with .items() and explain why .get() is safer.

Run the dictionaries mini-challenge (roommate rent) before moving on.

Step 7 - Sets and Set Operations

What to Say

"A set keeps unique items only, with no guaranteed order. Duplicates vanish automatically. Sets are also the quick way to de-duplicate a list and to compare groups."

Live Code

```
mumbai_visitors = {"Priya", "Arjun", "Priya", "Riya"}
print(mumbai_visitors)
# {'Priya', 'Arjun', 'Riya'}        # order may vary

unique_cities = set(["Mumbai", "Delhi", "Mumbai", "Pune", "Delhi"])

delhi_visitors = {"Riya", "Sanjay"}
print(mumbai_visitors | delhi_visitors)   # union
print(mumbai_visitors & delhi_visitors)   # intersection -> {'Riya'}
print(mumbai_visitors - delhi_visitors)   # difference
print(mumbai_visitors ^ delhi_visitors)   # symmetric difference

mumbai_visitors.add("Kavita")
mumbai_visitors.discard("Arjun")     # no error if absent
```

Common Errors

Error / symptom	Cause	Fix
Set prints in a different order each run	Sets are unordered by design	Use sorted(my_set) when order matters
{} is a dict, not a set	Empty braces make a dictionary	Use set() for an empty set
KeyError from remove()	remove() raises if item is absent	Use discard() for a safe remove

Done When

Students can name the four set operators and say what each returns.

Step 8 - Comprehensions

What to Say

"A comprehension builds a new collection from an existing one in a single line. First write the loop, then collapse it. The pattern is: [expression for item in iterable if condition]."

Live Code

```
# The long way
squares_loop = []
for n in range(1, 6):
    squares_loop.append(n ** 2)

# Comprehension: same result
squares = [n ** 2 for n in range(1, 6)]
# [1, 4, 9, 16, 25]

prices = [120, 560, 45, 899, 300, 610]
expensive = [p for p in prices if p > 500]
# [560, 899, 610]

price_in_rupees = {"tea": 15, "coffee": 25, "samosa": 20}
doubled = {item: p * 2 for item, p in price_in_rupees.items()}
even_set = {n for n in range(10) if n % 2 == 0}

# Mini-challenge answer pattern
results = ["Pass" if s >= 40 else "Fail" for s in [55, 32, 78, 40, 25]]
# ['Pass', 'Fail', 'Pass', 'Pass', 'Fail']
```

Common Errors

Error / symptom	Cause	Fix
Comprehension unreadable	Too many conditions on one line	If it will not fit one readable line, use a loop
Wrong brackets	[] list, {} set, {k: v} dict	Match the brackets to the type you want
Pass/Fail puts the if in the wrong place	Choosing a value needs if/else BEFORE the for	[a if cond else b for x in xs]

Done When

Students can rewrite a simple for loop as a comprehension.

Run the sets/comprehensions mini-challenge (Pass/Fail) to close.

SECTION 4 - SESSION WRAP-UP

Student Checklist

- [ ] I can create lists and use append, extend, insert, remove and pop
- [ ] I know a plain assignment does not copy a list
- [ ] I can slice a list or string with start:stop:step
- [ ] I can unpack a tuple and explain why it cannot change
- [ ] I can add, update, delete and safely look up dictionary keys
- [ ] I can use set union, intersection and difference
- [ ] I can write a list comprehension with an if condition
- [ ] I finished all four mini-challenges

Key Concepts Recap

Type	Syntax	Ordered	Changeable	Duplicates
list	[1, 2, 3]	Yes	Yes	Allowed
tuple	(1, 2, 3)	Yes	No	Allowed
dict	{'k': v}	Yes (insertion)	Yes	Keys must be unique
set	{1, 2, 3}	No	Yes	Not allowed

Day 5 Preview

Next session moves on to Day 5 of the course plan: Functions and Scope. Students will define their own functions, which they already saw once in the tuples file, and learn how Python decides where a variable lives.

Troubleshooting Notes

Nested list after append: the student passed a list to append(); use extend().

KeyError: the key is missing; use .get() or check key in d.

Original list changed: reference copy; use .copy().

Set order differs from the slide: expected; sets are unordered.

Tuple with one item is a string: add the trailing comma.

SECTION 5 - QUICK REFERENCE CARD

Essential Methods

Type	Method / syntax	What it does
list	lst.append(x)	Add x to the end
list	lst.extend(other)	Add every item of other
list	lst.insert(i, x)	Insert x at index i
list	lst.remove(x)	Remove first x (by value)
list	lst.pop()	Remove and return last item
list	lst.sort() / sorted(lst)	Sort in place / return new list
list	lst.copy() or lst[:]	Real copy
list	lst[start:stop:step]	Slice (stop exclusive)
tuple	a, b = (1, 2)	Unpack; use *rest for the remainder
dict	d[key] = v	Add or update
dict	d.get(key, default)	Safe lookup
dict	d.pop(key) / del d[key]	Remove a key
dict	d.items() / .keys() / .values()	Iterate pairs / keys / values
dict	d.update(other)	Merge another dict in
set	s.add(x) / s.discard(x)	Add / safe remove
set	s | t, s & t, s - t, s ^ t	Union, intersection, difference, symmetric difference
all	len(x), x in y	Length, membership test

Comprehension Cheat-Sheet

```
[expr for item in iterable]                 # list
[expr for item in iterable if condition]    # list with filter
[a if cond else b for item in iterable]     # list with if/else
{key: value for item in iterable}           # dict
{expr for item in iterable}                 # set
```

Choosing the Right Structure

If you need...	Use
An ordered, changeable collection	list
Fixed data, or several values returned from a function	tuple
Look up a value by a name or key	dict
Unique items, or to compare two groups	set